MAZEHUB

A maze gallery, built and shipped the DevOps way

Session 21 capstone. Ten modules, from the application to AWS.

Bibek Jyoti Charah . 24bcs10112

github.com/SammyUrfen/devops-session-21-final-project

MODULE 1

What the application does

A player picks a maze from the gallery and walks it on a canvas with the arrow keys.

The server carves every generated maze from a seed with a randomized depth-first search.

A breadth-first search proves the maze is solvable before the server stores it.

A second mode lets a person draw a maze by hand, wall by wall.

The server repeats five structural checks on a drawn grid, so no client can store a bad maze.

The browser records a run when the player reaches the exit cell. No sign in.

The data

2 tables, mazes and completions, with a cascading foreign key.

2 Alembic migrations. 14 REST endpoints.

The guard

Size, cell value, a closed border,

neighbours that agree about a shared wall,

and a real path from the entrance to the exit.

ARCHITECTURE

One request, end to end

```
Browser  ->  Ingress (nginx)  ->  /      ->  frontend Service  ->  2 nginx pods
                               \-> /api  ->  backend Service   ->  2 FastAPI pods

backend pod  ->  postgres Service  ->  postgres pod  ->  PersistentVolumeClaim
backend pod  ->  /metrics  <-  ServiceMonitor  <-  Prometheus  ->  Grafana
```

Control

Helm renders every object.

One chart, two value files.

Data

Postgres 16 on a PVC.

Alembic runs before uvicorn.

Scale

HPA holds the backend

between 2 and 5 pods at 50% CPU.

TECHNOLOGIES

The stack, and why each piece is there

Application

FastAPI and SQLAlchemy 2 on Python 3.12.
Alembic for the schema. Postgres 16.
React 19 and Vite. A canvas, no game library.

Delivery

Docker and docker compose.
GitHub Actions, 6 jobs. GHCR for the images.
Helm 3 and Kubernetes 1.31.

Security

Bandit for the Python source.
pip-audit and npm audit for the dependencies.
gitleaks over the history. Trivy on both images.

Platform

Terraform 1.16 for the AWS layer.
kube-prometheus-stack for the metrics.
Grafana for the dashboard.

MODULE 2

Tests come before the code

49 tests in total. 39 with pytest on the backend, 10 with node --test on the maze rules.

Each test runs against a throwaway SQLite file, never against Postgres.

The tests cover the carve, the solver, the completion guard and all five drawn-maze rules.

The pipeline runs both suites before it builds an image.

```
collected 39 items                                    [100%]
39 passed, 1 warning in 2.30s
tests 10   pass 10   fail 0
```

The cheat guard is the test I like most: a run with fewer moves than the shortest path gets a 422.

MODULE 4

Two images, both small and both non-root

backend

python:3.12-alpine.

Runs as uid 10001.

Command: alembic upgrade head, then uvicorn.

It waits for the Postgres health check first.

frontend

A 2 stage build.

Stage 1 is node:22-alpine and runs the Vite build.

Stage 2 is nginx-unprivileged and serves /dist.

Runs as uid 101.

```
$ docker compose up --build
  Image mazehub-frontend  Built      Container mazehub-postgres-1  Healthy
  Image mazehub-backend   Built      Uvicorn running on http://0.0.0.0:8000
```

MODULE 5

Six jobs, and the gate between them



Jobs 1 to 4 run at the same time. Job 5 declares needs on all four, so a red scanner blocks the push.

Job 5 tags both images with the commit SHA and pushes them to GHCR. Both packages are public.

Job 6 creates a kind cluster, installs the same Helm chart with those SHA tags, and runs a smoke test.

MODULE 6

Five scanners, one gate

Trivy scans both built images for OS packages, language packages and secrets.

The gate fails the job on a fixable HIGH or CRITICAL. It does not fail on a MEDIUM.

The last run found 1 MEDIUM and 0 HIGH or CRITICAL.

```
ghcr.io/sammyurfen/mazehub-backend:3db8615...  (alpine 3.24.2)
Total: 1 (LOW: 0, MEDIUM: 1, HIGH: 0, CRITICAL: 0)
zlib  CVE-2026-85091  MEDIUM  fixed  1.3.1-r0 -> 1.3.2-r1  heap buffer overflow
ghcr.io/sammyurfen/mazehub-frontend:3db8615...  0 vulnerabilities
```

The finding is real but not reachable: MazeHub never decompresses untrusted input. The base image rebuild clears it.

MODULE 8

Kubernetes and Helm

Workloads

2 backend pods.

2 frontend pods.

1 postgres pod on a PVC.

Network

3 ClusterIP Services.

1 Ingress: / to the frontend,

/api to the backend.

Health

Readiness and liveness probes.

An HPA from 2 to 5 pods.

Every pod Running.

```
$ helm upgrade --install mazehub helm/mazehub -n mazehub -f k8s/values-local.yaml
  STATUS: deployed   REVISION: 5
$ kubectl get pods -n mazehub
  5 pods, all 1/1 Running
```

MODULE 9

You cannot operate what you cannot see

The backend exposes /metrics in Prometheus format through a FastAPI instrumentator.

A ServiceMonitor from the chart tells kube-prometheus-stack where to look.

Prometheus reports the job mazehub-backend as 2 of 2 targets up.

The Grafana dashboard has five panels, and each one returns data.

The five panels

Requests per second by handler . p95 latency . Responses per second by status class
Backend targets up . Requests served in total
The last capture read 91284 requests served and both targets up.

MODULE 7

The AWS layer, created and destroyed by code

62 resources in one apply: a VPC on 10.21.0.0/16, 2 public and 2 private subnets,

a NAT gateway, an EKS cluster on 1.31, a managed node group of 2 t3.medium nodes, and 2 ECR repositories.

Both worker nodes joined and turned Ready in about 2 minutes.

```
Plan:    62 to add, 0 to change, 0 to destroy.
Apply complete! Resources: 62 added, 0 changed, 0 destroyed.
Destroy complete! Resources: 62 destroyed.
```

The window ran 2.5 hours and cost about USD 0.60. A check after the destroy found 0 clusters, 0 Elastic IPs and 0 ECR repositories. The account is clean.

MODULE 10

Commit, pipeline, deployment

The recording in demo/ shows the whole loop in 4 minutes.

The loop

0:05	One line changes in helm/mazehub/values.yaml.
0:33	git add, git commit, git push.
1:03	The pipeline starts. Six jobs.
2:33	Jobs 1 to 5 pass.
4:03	Job 6 installs the chart with the new SHA tags. 5 pods Running, the smoke test returns READY.

Every screenshot the rubric asks for is in docs/evidence/, with an index that names the module and the file.

HONEST LIMITS

What this project does not prove

The cluster evidence comes from minikube and from kind inside the runner. MazeHub never ran on EKS.

The AWS window was 2.5 hours. Nothing here proves the infrastructure survives a long run or an upgrade.

Terraform state is a local file. That suits one operator and would not suit a team.

The HPA was never put under a real load test. The 50% CPU target is a default, not a measured number.

There is no sign in. The player name lives in the browser, so the author field is a label, not an identity.

WHERE TO LOOK

Module, code, evidence

M1	Application	backend/app, frontend/src	docs/evidence/m01-*
M2	Tests	backend/tests	docs/evidence/m02-*
M3	Git	the commit history	docs/evidence/m03-*
M4	Docker	backend/ and frontend/Dockerfile	docs/evidence/m04-*
M5	CI/CD	.github/workflows/ci-cd.yml	docs/evidence/m05-*
M6	Trivy	security/TRIVY.md	docs/evidence/m06-*
M7	Terraform	terraform/	docs/evidence/m07-*
M8	Kubernetes	k8s/ and helm/mazehub/	docs/evidence/m08-*
M9	Observability	monitoring/	docs/evidence/m09-*
M10	Documentation	README.md	demo/

github.com/SammyUrfen/devops-session-21-final-project